

Лабораторная №6. Подготовка носителя и запись образа

Цель работы

Подготовить загрузочную SD-карту для Lichee RV Dock, проверить размещение компонентов до первого запуска и подтвердить по UART полную цепочку загрузки: BootROM, SPL, OpenSBI, U-Boot, ядро Linux и пользовательское пространство ALT Linux.

Результаты обучения

После выполнения работы студент должен уметь:

- однозначно отличать SD-карту от системного диска по пути, модели, размеру и серийному номеру;
- рассчитывать размещение MBR, U-Boot и разделов и доказывать отсутствие пересечений;
- создавать разделы BOOT/FAT32 и ROOTFS/ext4 с заданными метками;
- переносить ядро, DTB, конфигурацию Extlinux и rootfs без потери метаданных;
- проверять записанные файлы, U-Boot и файловые системы до извлечения карты;
- локализовать отказ по журналу UART без повторного форматирования карты.

Оборудование и исходные данные

- одноплатный компьютер Lichee RV Dock;
- SD-карта объёмом не менее 8 ГиБ и кардридер;
- компьютер с Linux и правами sudo;
- u-boot-sunxi-with-spl.bin для Lichee RV Dock;
- ядро kernels/Image_6.16_wifi_rt;
- DTB dts_and_dtb/sun20i-d1-lichee-rv-dock.dtb;
- архив ALT Regular rootfs для riscv64;
- USB-UART-преобразователь с логическими уровнями 3,3 В;
- утилиты lsblk, fdisk, blkid, mkfs.vfat, mkfs.ext4, tar, sha256sum, cmp, fsck.vfat и e2fsck.

Опасность повреждения

Команды dd, fdisk и mkfs безвозвратно уничтожают данные на выбранном устройстве. Перед каждой разрушающей командой заново проверьте путь, модель, размер и серийный номер карты. Не подставляй-

те /dev/sdX из примера без проверки.

Теоретические сведения

Цепочка загрузки

Загрузка используемого комплекта проходит последовательно:

1. BootROM процессора Allwinner D1 читает SPL с SD-карты.
2. SPL инициализирует DRAM и передаёт управление OpenSBI и U-Boot proper.
3. OpenSBI остаётся в M-mode и предоставляет ядру стандартный интерфейс SBI.
4. U-Boot читает extlinux/extlinux.conf из раздела BOOT, загружает Image и DTB в оперативную память.
5. Ядро использует DTB для описания оборудования, обнаруживает SD-карту и монтирует ROOTFS.
6. Ядро запускает /sbin/init, после чего systemd поднимает пользовательское пространство.

Ошибка на каждом уровне имеет свои признаки. Отсутствие баннера SPL указывает на проблему до U-Boot. Сообщение Found /extlinux/extlinux.conf подтверждает чтение BOOT. Строки Linux version, Machine model, Mounted root и Run /sbin/init последовательно подтверждают запуск ядра, правильный DTB, ROOTFS и init.

Размещение данных на карте

В работе используется таблица разделов MBR и следующая схема:

Область	Начало	Назначение
MBR	байт 0	таблица разделов
U-Boot SPL + proper	байт 8192	загрузчик Allwinner D1
BOOT	сектор 2048, размер 256 МиБ	FAT32, ядро, DTB, Extlinux
ROOTFS	сразу после BOOT	ext4, корневая файловая с

Первый раздел начинается с байта $2048 \times 512 = 1048576$. Следовательно, для U-Boot доступно $1048576 - 8192 = 1040384$ байта. Перед записью нужно сравнить этот объём с фактическим размером загрузчика. Для проверенного файла размером 956329 байт запас составляет 84055 байт.

Пространство SD-карты



Рис. 1 – Схема разбивки SD-карты на разделы BOOT и ROOTFS

Назначение файловых систем

Раздел BOOT форматируется в FAT32, которую умеет читать U-Boot. Раздел ROOTFS форматируется в ext4, поскольку корневая файловая система Linux должна сохранять владельцев, права, символические ссылки и специальные файлы устройств.

VFAT не хранит Unix-права. Поэтому rootfs нельзя распаковывать на BOOT или переносить через промежуточную FAT-файловую систему.

Согласованность комплекта

Ядро, DTB и внешние модули должны происходить из согласованной сборки. В проверенном учебном комплекте ядро имеет версию 6.16.0, а выданный rootfs содержит каталог модулей другой версии. Загрузка возможна, поскольку UART, MMC, ext4, FAT и Wi-Fi RTL8723DS встроены в ядро, однако внешние модули другой версии загружать нельзя. Для эксплуатационного образа следует установить `/lib/modules/$(uname -r)` из той же сборки, что и Image.

U-Boot из проверенного комплекта включает аппаратный watchdog с тайм-аутом 16 секунд. Ядро начинает обслуживать его примерно через 12 секунд после Starting kernel. Поэтому задержка timeout 50 в `extlinux.conf` расходует запас времени и приводит к циклическому перезапуску сразу после Starting kernel. В итоговой конфигурации ожидание меню не используется.

Порядок выполнения

1. Подготовка переменных

Задайте пути к исходным файлам. Путь `/dev/sdX` пока не изменяйте:

```
CARD=/dev/sdX
```

```
UBOOT=/путь/до/u-boot-sunxi-with-spl.bin
```

```
IMAGE=/путь/до/kernels/Image_6.16_wifi_rt
DTB=/путь/до/dts_and_dtb/sun20i-d1-lichee-rv-dock.dtb
ROOTFS_ARCHIVE=/путь/до/regular-rootfs-riscv64.tar.xz
```

Убедитесь, что исходные файлы существуют и имеют ненулевой размер:

```
stat -c '%n | %s байт' "$UBOOT" "$IMAGE" "$DTB" "$ROOTFS_ARCHIVE"
sha256sum "$UBOOT" "$IMAGE" "$DTB" "$ROOTFS_ARCHIVE"
```

Сохраните контрольные суммы в отчёте. Они позволяют отличить повреждение файла от ошибки разметки или конфигурации.

2. Однозначная идентификация SD-карты

Сравните список устройств до и после подключения кардридера:

```
lsblk -p -o NAME,TYPE,SIZE,MODEL,SERIAL,TRAN,FSTYPE,LABEL,MOUNTPOINTS
```

После проверки замените значение CARD фактическим путём, например:

```
CARD=/dev/sdb
```

Получите дополнительное подтверждение:

```
lsblk -p -o NAME,TYPE,SIZE,MODEL,SERIAL,TRAN,FSTYPE,LABEL,MOUNTPOINTS "$CARD"
sudo fdisk -l "$CARD"
findmnt -rn --source "${CARD}1"
findmnt -rn --source "${CARD}2"
```

Запишите в отчёт путь, модель, серийный номер и размер в байтах. Сравните их с системным диском.

Опасность повреждения

До продолжения преподаватель или напарник должен подтвердить, что CARD указывает именно на SD-карту. Если модель или размер вызывают сомнения, остановите работу.

Размонтируйте разделы, если они подключены:

```
sudo umount "${CARD}1" "${CARD}2" 2>/dev/null || true
```

Повторно убедитесь, что точек монтирования нет:

```
lsblk -p -o NAME,SIZE,MODEL,SERIAL,MOUNTPOINTS "$CARD"
```

3. Резервная копия начала карты

Сохраните первые 10 МиБ старого носителя в каталог на системном диске:

```
sudo dd if="$CARD" of=sdcard-first-10MiB-before.bin \
    bs=1M count=10 iflag=fullblock status=progress
sudo chown "$(id -u):$(id -g)" sdcard-first-10MiB-before.bin
sha256sum sdcard-first-10MiB-before.bin
```

Эта копия сохраняет MBR, старый U-Boot и начало первого раздела. Она не заменяет резервную копию всех пользовательских данных карты.

4. Очистка и создание разделов

После последней проверки устройства очистите первые 10 МиБ:

```
sudo dd if=/dev/zero of="$CARD" bs=1M count=10 status=progress conv=fsync
```

Запустите fdisk:

```
sudo fdisk "$CARD"
```

Последовательно введите:

```
o          # новая таблица DOS/MBR
n          # новый раздел
p          # основной
1          # номер 1
2048       # первый сектор
+256M      # размер B00T
t          # изменить тип раздела 1
c          # W95 FAT32 (LBA), код 0x0c
n          # новый раздел
p          # основной
2          # номер 2
<Enter>    # первый предложенный сектор
<Enter>    # до конца карты
w          # записать таблицу
```

Попросите ядро перечитать таблицу и проверьте результат:

```
sudo partprobe "$CARD"
sudo udevadm settle
sudo fdisk -l "$CARD"
lsblk -p -o NAME,START,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINTS "$CARD"
```

Ожидается MBR, первый раздел от сектора 2048 размером 256 МиБ и второй раздел на оставшемся пространстве.

5. Создание файловых систем

```
sudo mkfs.vfat -F 32 -n BOOT "${CARD}1"
sudo mkfs.ext4 -L ROOTFS "${CARD}2"
sudo blkid "${CARD}1" "${CARD}2"
```

Ожидаются TYPE="vfat", LABEL="BOOT", TYPE="ext4" и LABEL="ROOTFS".

6. Проверка свободной области и запись U-Boot

Вычислите доступное место перед первым разделом:

```
UBOOT_SIZE=$(stat -c %s "$UBOOT")
FIRST_SECTOR=$(lsblk -dnro START "${CARD}1")
SECTOR_SIZE=$(sudo blockdev --getss "${CARD}")
UBOOT_OFFSET=8192
AVAILABLE=$((FIRST_SECTOR * SECTOR_SIZE - UBOOT_OFFSET))
printf 'U-Boot: %s байт; доступно: %s байт\n' "$UBOOT_SIZE" "$AVAILABLE"
test "$UBOOT_SIZE" -le "$AVAILABLE"
```

Команда test должна завершиться с кодом 0. Запишите загрузчик на всё устройство, а не в раздел:

```
sudo dd if="$UBOOT" of="$CARD" bs=1K seek=8 \
conv=fsync,notrunc status=progress
```

Прочитайте записанные байты обратно и сравните с исходником:

```
sudo dd if="$CARD" iflag=skip_bytes,count_bytes \
skip=8192 count="$UBOOT_SIZE" status=none | \
cmp -n "$UBOOT_SIZE" "$UBOOT" -
```

Отсутствие вывода и код возврата 0 означают побайтовое совпадение.

7. Монтирование разделов

```
sudo mkdir -p /mnt/lab6-boot /mnt/lab6-rootfs
sudo mount "${CARD}1" /mnt/lab6-boot
sudo mount "${CARD}2" /mnt/lab6-rootfs
findmnt -T /mnt/lab6-boot
findmnt -T /mnt/lab6-rootfs
```

8. Заполнение BOOT

```
sudo install -m 0644 "$IMAGE" /mnt/lab6-boot/Image
sudo install -m 0644 "$DTB" /mnt/lab6-boot/sun20i-d1-lichee-rv-dock.dtb
sudo mkdir -p /mnt/lab6-boot/extlinux
sudo tee /mnt/lab6-boot/extlinux/extlinux.conf >/dev/null <<'EOF'
label Linux
kernel /Image
fdt /sun20i-d1-lichee-rv-dock.dtb
append root=/dev/mmcblk0p2 rootwait loglevel=7 console=ttyS0,115200
EOF
```

Внимание

Не добавляйте параметр `timeout` в эту конфигурацию. На проверенном U-Boot ожидание меню вместе с поздней инициализацией watchdog-драйвера ядра вызывает циклический сброс после `Starting kernel`.

В данной схеме ядро запускается без `initramfs`, поэтому используется проверенный путь `/dev/mmcblk0p2`. Запись `root=LABEL=R00TFS` требует механизма разрешения метки в `initramfs` и не должна применяться без отдельной проверки.

Проверьте структуру:

```
find /mnt/lab6-boot -maxdepth 2 -type f -printf '%P | %s байт\n'
cat /mnt/lab6-boot/extlinux/extlinux.conf
```

9. Заполнение ROOTFS

Проверьте архив и распакуйте его с числовыми владельцами:

```
xz -t "$R00TFS_ARCHIVE"
sudo tar --numeric-owner -xJpf "$R00TFS_ARCHIVE" -C /mnt/lab6-rootfs
```

Проверьте `init`, архитектуру `systemd` и специальные файлы:

```
sudo readlink /mnt/lab6-rootfs/usr/sbin/init
file /mnt/lab6-rootfs/usr/lib/systemd/systemd
sudo find /mnt/lab6-rootfs/dev -maxdepth 1 \
    \( -type b -o -type c \) -printf '%y %p %u:%g\n'
```

Настройте `/etc/fstab`:

```
sudo tee /mnt/lab6-rootfs/etc/fstab >/dev/null <<'EOF'
proc                /proc                proc    nosuid,noexec,gid=proc
↪                0 0
devpts              /dev/pts                devpts
↪  nosuid,noexec,gid=tty,mode=620,ptmxmode=0666        0 0
tmpfs               /tmp                tmpfs    nosuid
↪                0 0
LABEL=R00TFS        /                ext4     defaults
↪                1 1
LABEL=B00T          /boot                vfat     defaults,noatime,icharset=cp437,nofail
↪                0 2
EOF
```

Параметр `icharset=cp437` использует таблицу NLS, встроенную в проверенное ядро. Без него VFAT запрашивает внешний модуль `nls_iso8859-1`, отсутствующий в `rootfs` другой версии. Параметр `nofail` не переводит

систему в аварийный режим при необязательной ошибке монтирования BOOT.

Используйте выданные преподавателем учётные данные. Если пароль root для учебного ALT rootfs заранее не задан, создайте временный пароль без помещения открытого текста в историю команд:

```
ROOT_SHADOW=/mnt/lab6-rootfs/etc/tcb/root/shadow
test -f "$ROOT_SHADOW"
ROOT_HASH=$(openssl passwd -6)
sudo cp -a "$ROOT_SHADOW" "${ROOT_SHADOW}.lab6-backup"
sudo sed -i "s|^root:[^:]*:|root:${ROOT_HASH}:|" "$ROOT_SHADOW"
sudo chown --reference="${ROOT_SHADOW}.lab6-backup" "$ROOT_SHADOW"
sudo chmod --reference="${ROOT_SHADOW}.lab6-backup" "$ROOT_SHADOW"
sudo rm "${ROOT_SHADOW}.lab6-backup"
unset ROOT_HASH
```

openssl запросит пароль интерактивно. После лабораторной временный пароль необходимо заменить. Для rootfs другого дистрибутива путь к shadow и механизм хранения паролей могут отличаться.

10. Проверка записанных данных

Сравните хеши исходных файлов и файлов на BOOT:

```
sha256sum "$IMAGE" /mnt/lab6-boot/Image
sha256sum "$DTB" /mnt/lab6-boot/sun20i-d1-lichee-rv-dock.dtb
```

Проверьте метки, владельцев и точки монтирования:

```
lsblk -p -o NAME,START,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS "$CARD"
sudo stat -c '%n | %U:%G | %a | %F' \
    /mnt/lab6-rootfs/usr/lib/systemd/systemd \
    /mnt/lab6-rootfs/etc/fstab
findmnt -T /mnt/lab6-boot
findmnt -T /mnt/lab6-rootfs
```

Сбросьте кэш и размонтируйте разделы:

```
sync
sudo umount /mnt/lab6-boot /mnt/lab6-rootfs
```

Выполните неразрушающую проверку файловых систем:

```
sudo fsck.vfat -n "${CARD}1"
sudo e2fsck -fn "${CARD}2"
lsblk -p -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINTS "$CARD"
```

Обе проверки должны завершиться без неисправленных ошибок, а точки монтирования должны отсутствовать. Только после этого извлеките карту.

11. Проверка загрузки по UART

Подключите UART к выключенной плате: GND к GND, TX преобразователя к RX платы, RX преобразователя к TX платы. Используйте уровень 3,3 В и не подключайте линию 5 В.

Откройте терминал со скоростью 115200 8N1 и журналированием:

```
tio -b 115200 -d 8 -s 1 -p none -f none \  
-t -L --log-file lab6-boot.log /dev/ttyUSBX
```

Подайте питание. В успешном журнале должны присутствовать:

```
U-Boot SPL  
OpenSBI  
Found /extlinux/extlinux.conf  
Starting kernel  
Linux version 6.16.0  
Machine model: Sipeed Lichee RV Dock  
mmcblk0: p1 p2  
VFS: Mounted root (ext4 filesystem)  
Run /sbin/init as init process  
Mounted boot.mount - /boot  
localhost login:
```

После входа выполните:

```
uname -a  
tr -d '\0' </proc/device-tree/model; echo  
cat /proc/cmdline  
findmnt / /boot  
ip -brief link show wlan0  
systemctl is-system-running  
systemctl --failed --no-pager  
ls /usr/lib/modules
```

Ожидаются riscv64, PREEMPT_RT, модель Sipeed Lichee RV Dock, ROOTFS на /dev/mmcblk0p2, BOOT на /dev/mmcblk0p1 и интерфейс wlan0. Служба smartd может завершиться ошибкой, поскольку обычная SD-карта не предоставляет SMART; это не означает ошибку загрузочного образа.

Диагностика неисправностей

Симптом	Вероятная причина	Минимальная проверка и исправление
Нет баннера SPL	U-Boot отсутствует, записан не на устройство или неверно смещение	Сравнить байты от смещения 8192 с исходным U-Boot
U-Boot не находит Extlinux	Неверный путь или файловая система BOOT	Проверить BOOT/extlinux/extlinux.conf и FAT32
Циклический сброс после Starting kernel	Тайм-аут меню исчерпал 16 секунд watchdog	Удалить timeout и повторить загрузку без переразметки
Waiting for root device	Неверный root= или драйвер MMC не встроен	Проверить /proc/cmdline, номер раздела и конфигурацию ядра
Machine model не соответствует плате	Загружен DTB другой платы	Проверить fdt и содержимое DTB через dtc
Emergency mode, IO charset iso8859-1 not found	Требуемый NLS собран модулем другой версии	Использовать iocharset=cp437,nofail или установить согласованные модули
Invalid module format	Версия внешнего модуля не совпадает с uname -r	Установить модули из той же сборки, что и ядро

Не начинайте диагностику с повторного dd, форматирования или распаковки rootfs. Сначала определите последний успешно завершённый слой загрузки.

Задание

1. Однозначно идентифицируйте SD-карту и сохраните сведения о старой разметке.
2. Создайте MBR, BOOT размером 256 МиБ и ROOTFS на оставшемся пространстве.
3. Докажите расчётом, что U-Boot помещается до первого раздела.
4. Запишите U-Boot и выполните побайтовое сравнение с исходным файлом.

5. Заполните BOOT и ROOTFS, сохранив владельцев, права и специальные файлы.
6. Проверьте хеши, метки, структуру, точки монтирования и файловые системы.
7. Выполните sync, размонтируйте карту и загрузите Lichee RV Dock.
8. Сохраните UART-журнал и отметьте в нём этапы цепочки загрузки.
9. После входа подтвердите RT-ядро, модель платы, оба раздела и Wi-Fi-интерфейс.

Дополнительные задания

Вариант 1. Паспорт и контрольная точка носителя

Составьте паспорт карты до первой разрушающей команды: путь, модель, серийный номер, размер в байтах, старая таблица разделов, точки монтирования и SHA-256 резервной копии первых 10 МиБ. Напарник должен письменно подтвердить выбранное устройство.

Вариант 2. Карта байтовых диапазонов

Получите начальный сектор BOOT, логический размер сектора и размер U-Boot. Вычислите начальный и конечный байты загрузчика и запас до первого раздела. Подтвердите вычисления фактическим чтением U-Boot с карты.

Вариант 3. Манифест записанного комплекта

Создайте таблицу SHA-256 исходных и записанных Image, DTB и extlinux.conf. Для U-Boot используйте чтение точного диапазона со смещения 8192. Все пары должны совпасть.

Вариант 4. Аудит метаданных ROOTFS

Выберите обычный исполняемый файл, символическую ссылку и специальный файл устройства. Зафиксируйте тип, владельца и режим после распаковки и объясните, почему обычное копирование через VFAT разрушило бы часть этих метаданных.

Вариант 5. Послойная диагностика загрузки

Аннотируйте UART-журнал: для SPL, OpenSBI, U-Boot, ядра, ROOTFS и init укажите первую строку, доказывающую успешность этапа. Для одного отсутствующего маркера предложите неразрушающую проверку.

Вариант 6. Исследование watchdog

Сравните загрузку конфигурации без timeout и с timeout 50. Измерьте время от Starting kernel до строки инициализации sunxi-wdt, объясните циклический сброс и восстановите рабочую конфигурацию без перерезметки и повторной записи U-Boot.

Контрольные вопросы

1. Почему BOOT и ROOTFS используют разные файловые системы?
2. Зачем U-Boot записывается со смещением 8 КиБ и как доказать отсутствие пересечения с BOOT?
3. Какие компоненты и параметры связывает extlinux.conf?
4. Почему rootfs распаковывается с --numeric-owner -xJpf?
5. Почему внешний модуль должен совпадать с версией работающего ядра?
6. Почему timeout 50 приводит к сбросу проверенного стенда после Starting kernel?
7. Чем проверка SHA-256 файлов BOOT отличается от побайтовой проверки U-Boot?
8. По каким строкам UART-журнала можно разделить отказ U-Boot, ядра, ROOTFS и init?

Требования к отчёту

Отчёт должен содержать:

- цель работы и сведения о стенде;
- паспорт SD-карты и подтверждение выбора устройства;
- старую и новую таблицы разделов, метки и UUID;
- расчёт размещения U-Boot и запаса до BOOT;
- версии, размеры и SHA-256 исходных компонентов;
- команды записи и результаты обратной проверки;
- итоговые extlinux.conf и /etc/fstab;
- структуру BOOT и примеры метаданных ROOTFS;
- результаты fsck.vfat -n и e2fsck -fn;
- аннотированный UART-журнал успешной загрузки;
- выводы команд проверки после входа;
- ответы на контрольные вопросы и краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.